Artificial Intelligence

Tut/Lab week 2

COSC2129/2713/3118

RMIT UNIVERSITY

Outline

- Search
  - Blind search
  - Informed search
  - Hands-on examples
- Assignment 1 part 1
- Q&A

What to expect?

Course Objectives

- AI is cross-disciplinary, covering a huge variety of subfields ranging from scientific research to real-world applications.
- This course only introduces to the basic concepts and techniques of AI, leaving advanced topics for self-exploration or the undertaking of advanced courses.
- This course will help you gain AI-based problem solving skills that have applicability to a wide range of real-world problems.
- Basic concepts (and techniques) lays foundations for the success of in-depth studies and even job interviews.

Potential job interview questions

- Give some real-world applications of AI
- What is an intelligent agent?
- What are the types of AI?
- What are the different domains/subsets of AI?
- Give a brief introduction to the Turing test in AI?
- Which programming language is used for AI?
- What is Strong AI, and how is it different from the Weak AI?
- What are some misconceptions about AI?
- …

Challenge & Update

- What is the most important event/archievement in AI last week (in your view, from your understanding)?

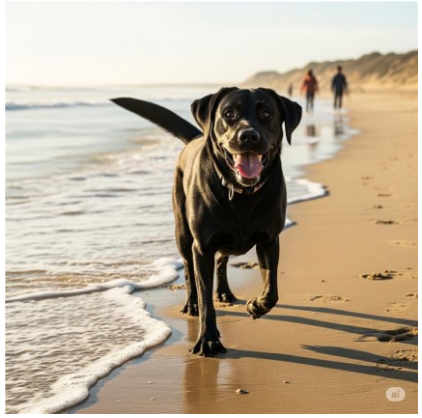
Challenge & Update (7/2025)

- What is the most important event/archievement in AI last week (in your view, from your understanding)?

  - This AI ‘thinks’ like a human

- after training on 160 psychology studies The Centaur model goes beyond single tasks and predicts a wide array of human behaviour.

  - When pixels replace paintbrushes: The Creative Industries



RMIT UNIVERSITY

Challenge & Update (2024)

- What is the most important event/archievement in AI last week (in your view, from your understanding)?
  - OpenAI's Search Revolution: ChatGPT Just Changed the Game Forever. ChatGPT can now search the web in a much better way than before
  - Meta's plan for nuclear-powered AI data centre thwarted by rare bees

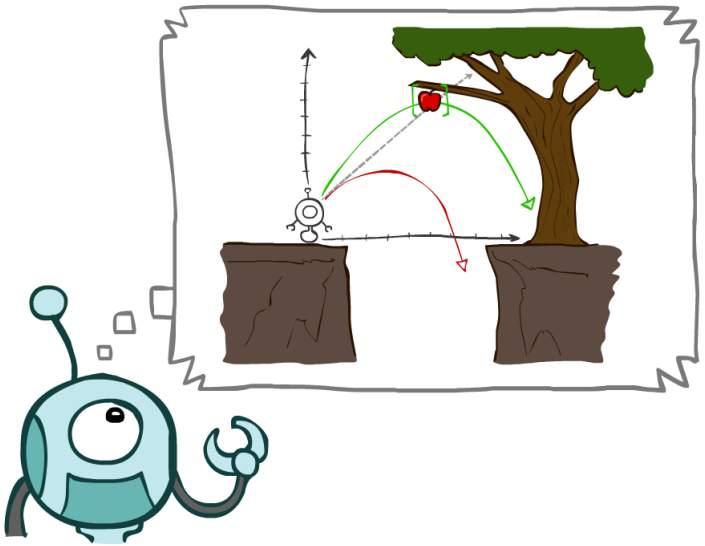
➢ What is the lession to learn?

Course material

- Book: Artificial Intelligence: A Modern Approach
- Book - TOC of Artificial Intelligence: A Modern Approach, 4th Global ed.
- AIMA: Pseudo code
- Good practices Python

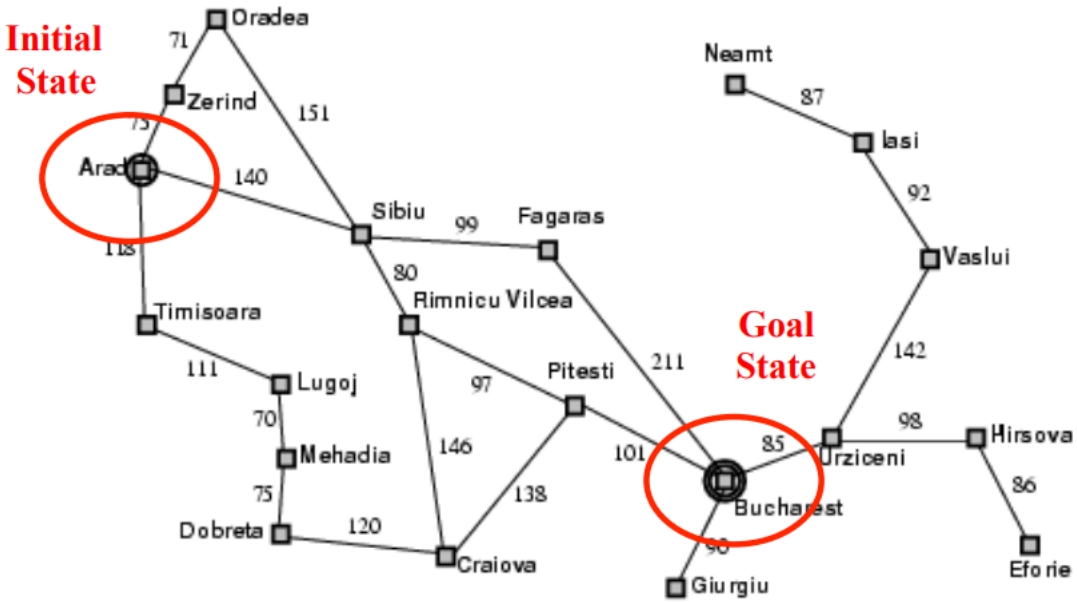
Activity 1

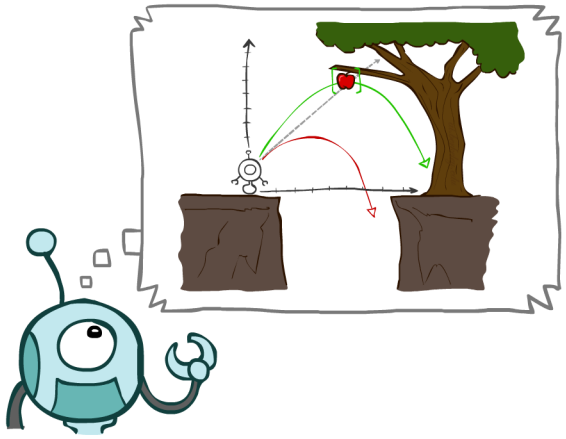
- What is Search?
  - Search is an exploration of possibilities



Why search?

- Path finding problem

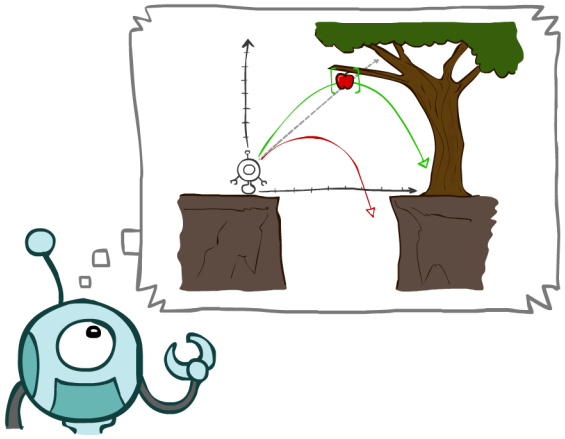


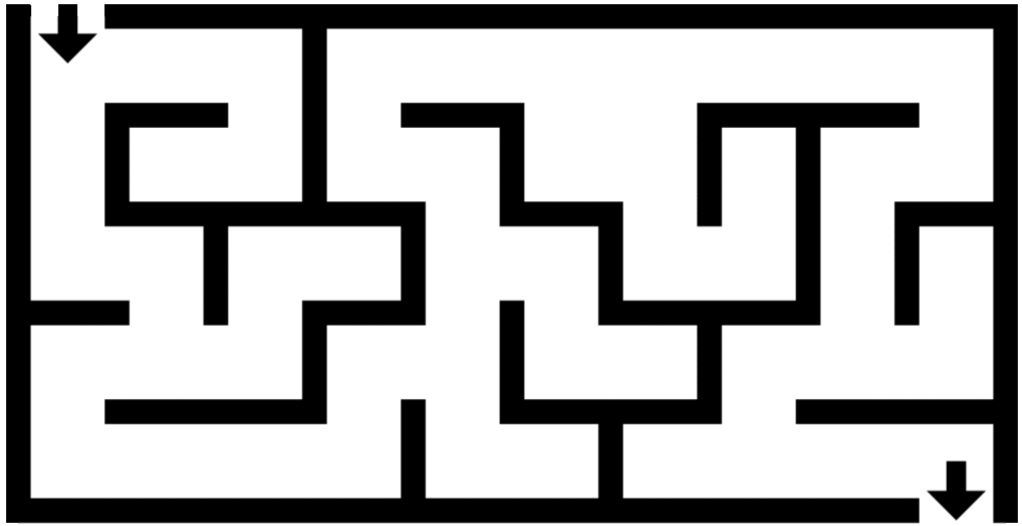


RMIT UNIVERSITY

Why search?

- Path finding problem

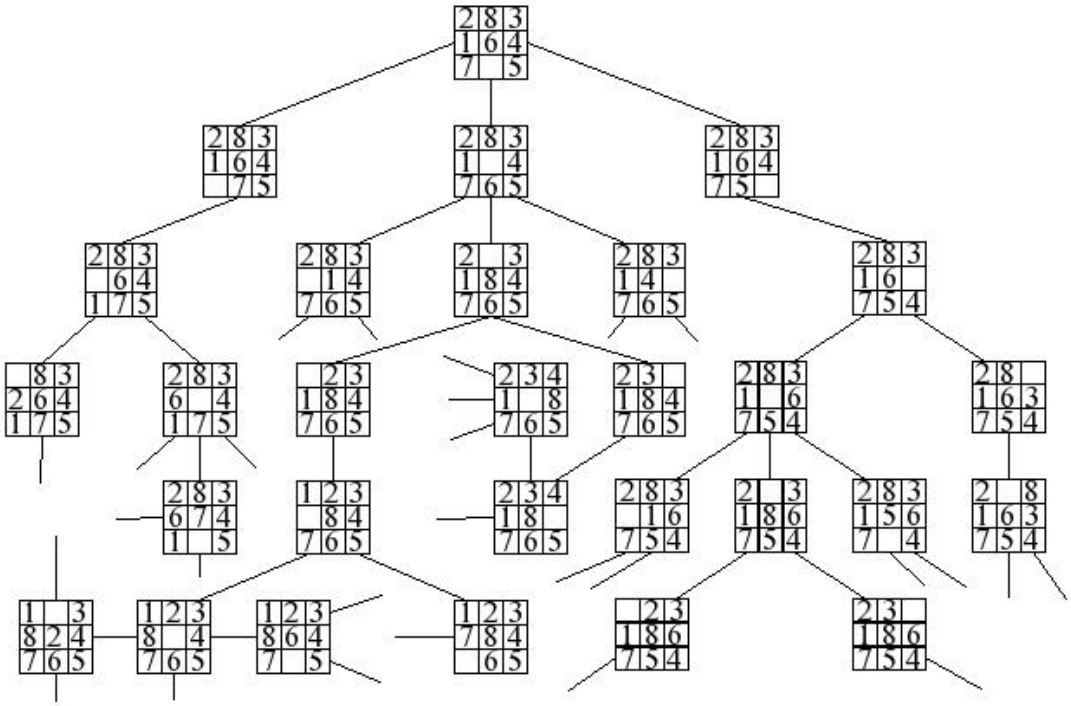




Why search

- 8 puzzle problem

8		6
5	4	7
2	3	1



	1	2
3	4	5
6	7	8

RMIT UNIVERSITY

Activity 1

- Give some examples of applications of problem solving by search.
  - Path finding
  - Robot navigation
  - VLSI design
  - Theorem proving
  - Game
  - …

Example explained

- VLSI (Very Large Scale Integration) layout can be considered a search problem in the context of computer-aided design (CAD) for integrated circuits.
- layout refers to the physical arrangement of components (such as transistors, capacitors, resistors, and interconnections) on a semiconductor chip.

RMIT UNIVERSITY

Automated theorem proving

- A set of axioms → Conclusion.
- Search within a space of possible logical steps, starting from a set of axioms (or premises) and following the rules of inference.

$$\dfrac{\dfrac{p \to (q \to r) \qquad \dfrac{[p \wedge q]}{p}\,\wedge_{e1}}{q \to r}\,\to_e \qquad \dfrac{[p \wedge q]}{q}\,\wedge_{e2}}{\dfrac{r}{p \wedge q \to r}\,\to_i}\,\to_e$$

RMIT UNIVERSITY

Real-Life Application: GPS navigation for driving

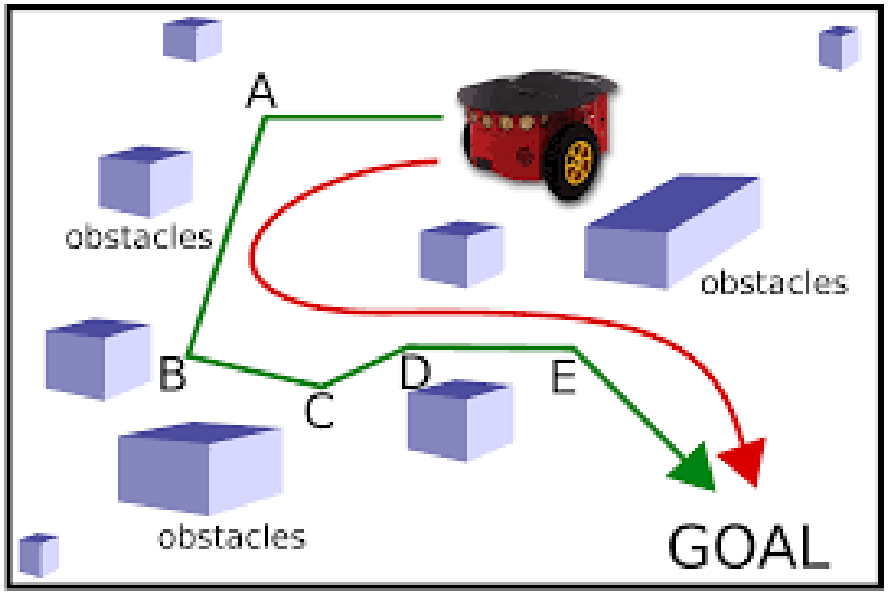
- Search problem setup: 5 components

  - ✓ **Initial state**: The start point is your current location, ex. Point A.
  - ✓ **Goal state**: The goal point is the destination, ex. Point B (the restaurant/home in path finding).
  - ✓ **State space**: The map consists of roads (nodes) and intersections (edges).
  - ✓ **Action**: What actions are possible in a given state.
  - ✓ **Transition model**: What happens when an action is applied to a state

Real-Life Application: GPS Navigation for Driving

- What are cost factors?
- What can be heuristics?
  - **g-cost:** the actual driving time or distance from your starting point to a given intersection.
  - **h-cost:** the heuristic estimate of the driving time or distance from the current intersection to your destination (using something like straight-line distance or traffic-based estimates).
  - Traffic and road conditions can influence both the g-cost and the h-cost.

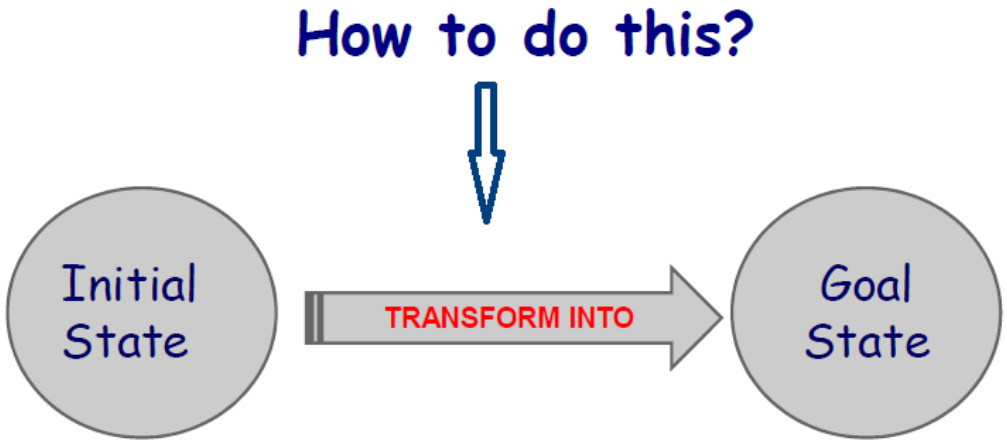
Problem solving by search



https://www.mdpi.com/1424-8220/18/9/3170

Search Problems

- A search problem is defined by:
  - Possible states (search space)
  - Initial state
  - Actions
  - Transition mode
  - Goal test
  - Path cost

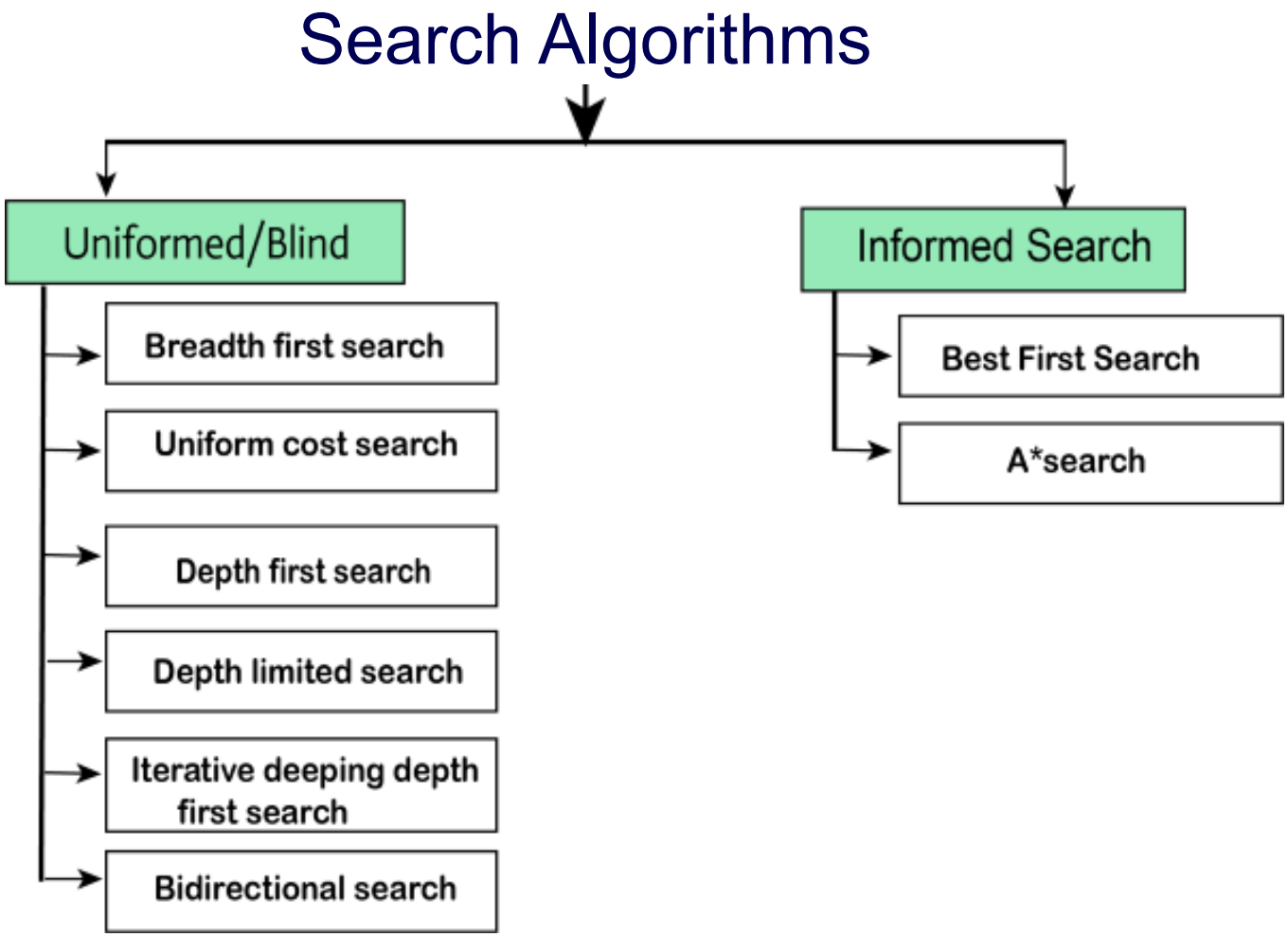


Activity 2

- What is a state for each of the example above?
- How to represent a state?

- What is a state space?
- How to represent it?

Search Algorithms



Performance of Search Strategies

- Completeness: Is it guaranteed to find a solution if one exists?
- Optimality: Will it find an optimal solution?
- Time complexity: How many nodes get expanded?
- Space complexity: How much memory is needed?

- Time and space complexity is calculated based on:
  - *b*: Branching factor
  - *d*: Depth of the shallowest goal node
  - *m*: Maximum length of any path in the state space

Lab

- To reinforce your understanding of basic search methods.
- To acquaint you with the operation and presentation of AI-Search software
- N-Puzzle: browse to the following address and open the AI-Search Software
  https://learninglab.rmit.edu.au/Toolbox/AI-Search/index.html
- Follow the instructions

Activity 3: Hand-on Practice

- Hand-on slides

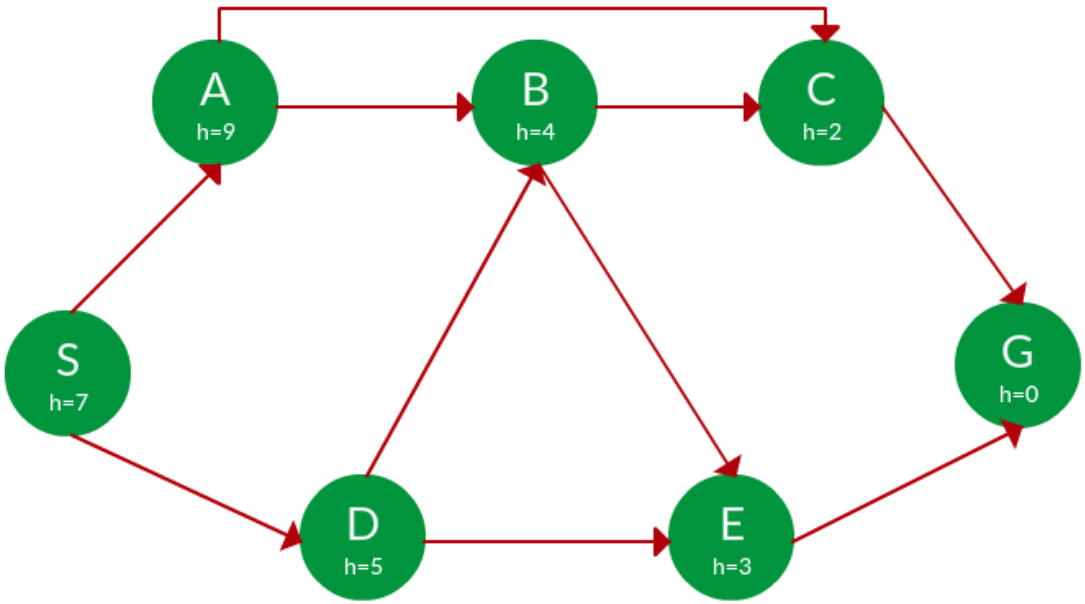
Practice

- Recap Greedy Search: Use heuristics to guide the search
- Heuristic:
  - A heuristic h is defined as h(x) = **Estimate** of distance of node x from the goal node.
  - Lower the value of h(x), closer is the node from the goal.
- Strategy: Expand the node closest to the goal state, i.e. expand the node with *a lower h value*.

Practice

- Greedy Search example

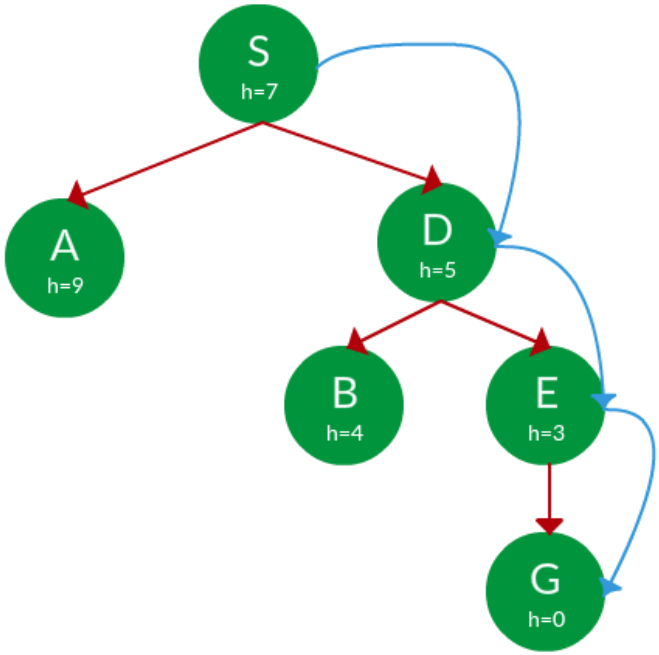
  Find the path from S to G using greedy search. The heuristic values h of each node below the name of the node.



https://www.geeksforgeeks.org/search-algorithms-in-ai/

Practice

- Solution
  - Start from S, we can traverse to A(h=9) or D(h=5). We choose D, as it has the lower heuristic cost.
  - From D, we can move to B(h=4) or E(h=3). We choose E with a lower heuristic cost.
  - From E, we go to G(h=0). This entire traversal is shown in the search tree below.



Practice

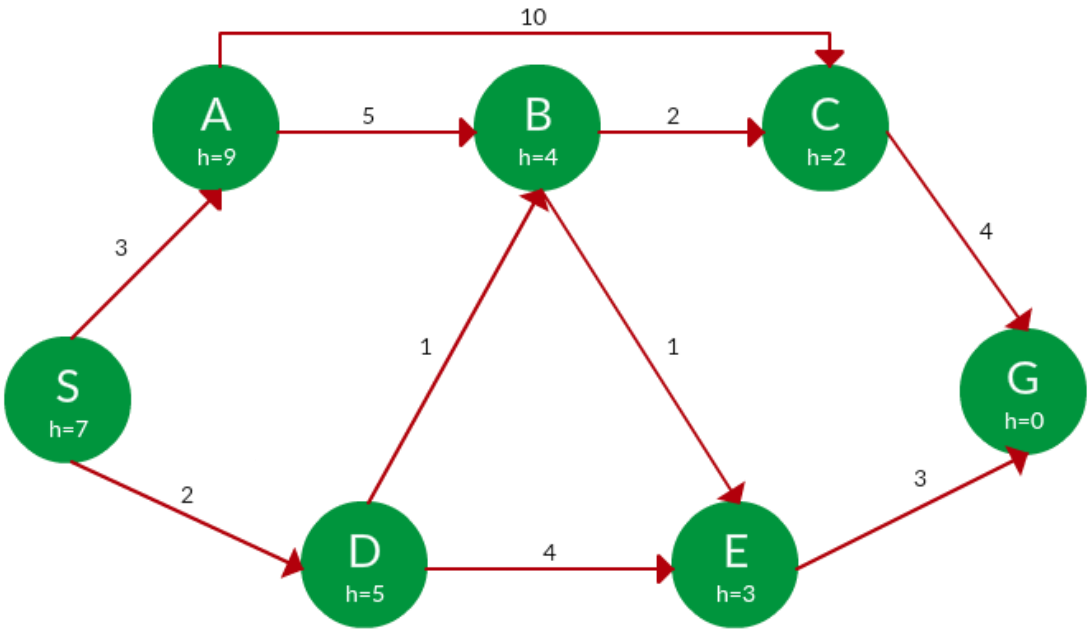
- A* Tree Search
  - Heuristic: **f(x) = g(x) + h(x)**
    - Here, **h(x)**, *the forward cost,* is an estimate of the distance of the current node from the goal node.
    - And, **g(x)**, *the backward cost,* is the cumulative cost of a node from the root node.
    - Admissibility:

$$0 \leqslant h(x) \leqslant h^*(x)$$

    - **Strategy**: Choose the node with the lowest f(x) value.

Practice

- A* search: **f(x) = g(x) + h(x)**



https://www.geeksforgeeks.org/search-algorithms-in-ai/

RMIT UNIVERSITY

Practice

- Solution

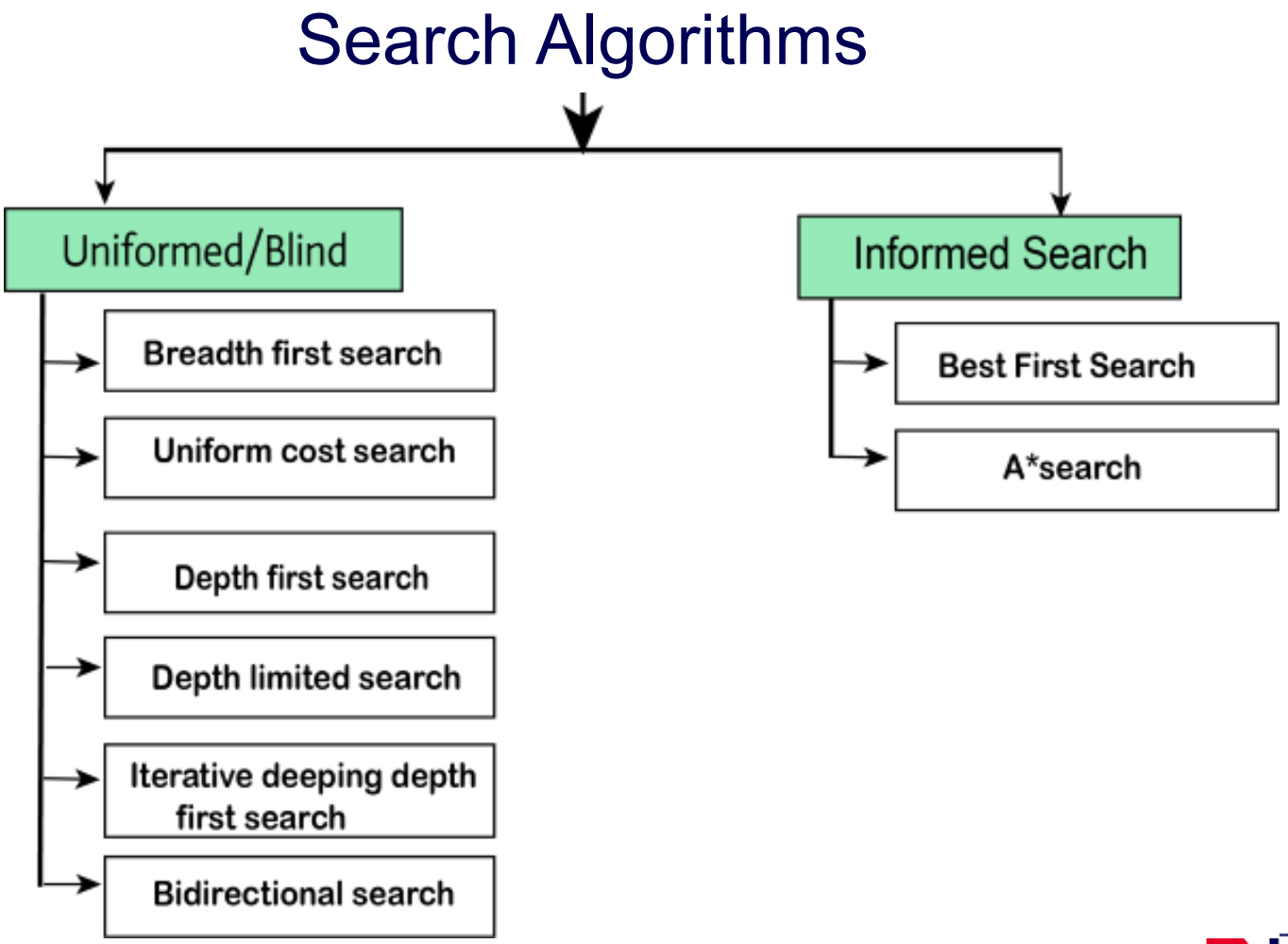
Path		h(x)	g(x)	f(x)
S		7	0	7
S -> A		9	3	12
S -> D	✓	5	2	7
S -> D -> B	✓	4	2 + 1 = 37	
S -> D -> E		3	2 + 4 = 69	
S -> D -> B -> C	✓	2	3 + 2 = 57	
S -> D -> B -> E	✓	3	3 + 1 = 47	
S -> D -> B -> C -> G		0	5 + 4 = 99	
S -> D -> B -> E -> G	✓	0	4 + 3 = 77	

Path: S -> D -> B -> E -> G

Cost: 7

RMIT UNIVERSITY

Recap


Search Algorithms
Uniformed/Blind
Breadth first search
Uniform cost search
Depth first search
Depth limited search
Iterative deeping depth first search
Bidirectional search
Informed Search
Best First Search
A*search

Q&A

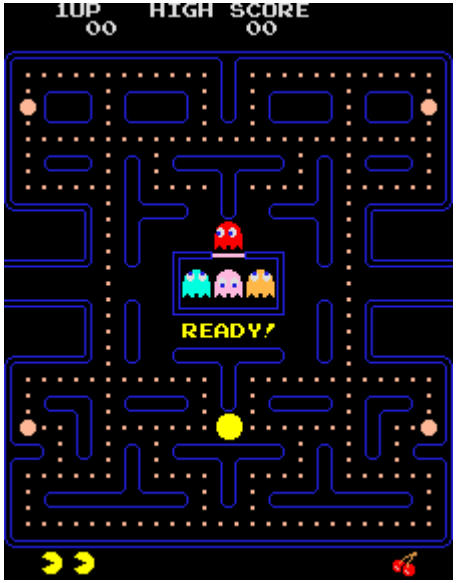
- Why A* is efficient?
- because it uses both the actual cost from the start node (g) and an estimated cost to the goal (h), so it doesn't just blindly explore all paths. The heuristic helps prioritize promising paths while still ensuring the algorithm finds the optimal solution.

Project 0

- CS188 Intro to AI - Course Materials
- Tutorial: Python, Set up & Autograder Tutorial

Activity 4

- PACMAN and Assignment 1 part 1



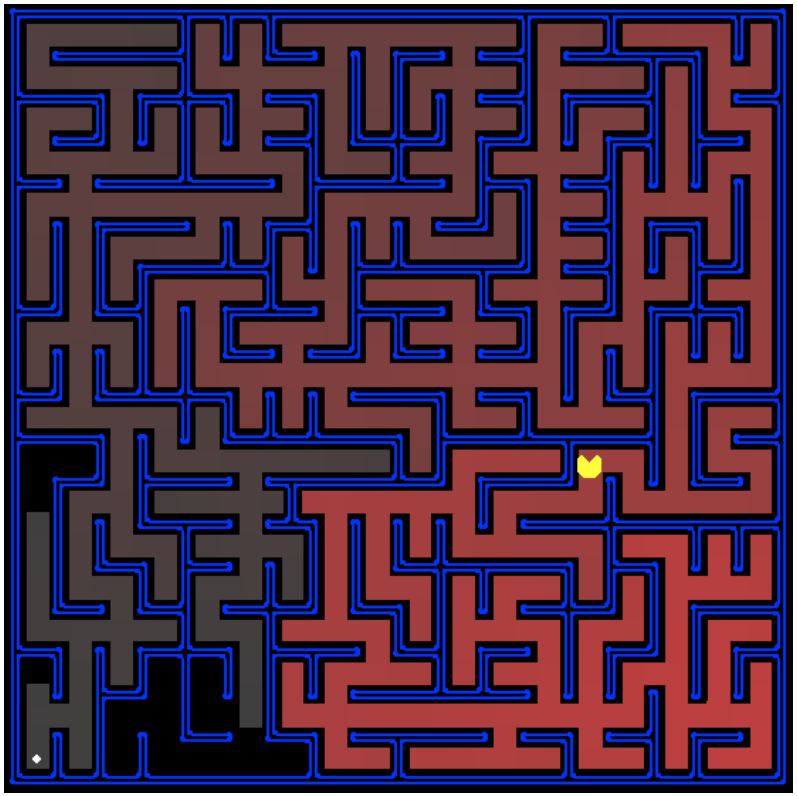
The game remains one of the highest-grossing and best-selling games, generating more than $14 billion in revenue (as of 2016) and 43 million units in sales combined, and has an enduring commercial and cultural legacy, commonly listed as one of the greatest video games of all time.

https://en.wikipedia.org/wiki/Pac-Man

RMIT UNIVERSITY

Assignment 1-1: Pacman game

- Pacman in A1-1
  - Question 1
  - Question 2
  - Question 3
  - Question 4
  - Question 5
  - Question 6
  - Question 7
  - Question 8



CS 188 UC Berkeley

Notes

Files you'll edit:

search.py	Where all of your search algorithms will reside.
searchAgents.py	Where all of your search-based agents will reside.

Files you might want to look at:

pacman.py	The main file that runs Pacman games. This file describes a Pacman GameState type, which you use in this project.
game.py	The logic behind how the Pacman world works. This file describes several supporting types like AgentState, Agent, Direction, and Grid.
util.py	Useful data structures for implementing search algorithms.

Supporting files you can ignore:

graphicsDisplay.py	Graphics for Pacman
graphicsUtils.py	Support for Pacman graphics
textDisplay.py	ASCII graphics for Pacman
ghostAgents.py	Agents to control ghosts
keyboardAgents.py	Keyboard interfaces to control Pacman

Notes

- **Hint**: Each algorithm is very similar. Algorithms for DFS, BFS, UCS, and A* differ only in the details of how the fringe is managed.
  - Concentrate on DFS, the rest should be relatively straightforward.
  - Implement the depth-first search (DFS) algorithm in the depthFirstSearch function in search.py.
  - To make your algorithm complete, write the graph search version of DFS, which avoids expanding any already visited states.

Question?

RMIT UNIVERSITY